

CS 641: Modern Cryptography

Homework 2 - Question 3 Report

Venkatesh (220109)

April 12, 2026

Part (a):

Statement: Prove that if a cyclic group has n elements, it contains exactly $\phi(n)$ generators.

Proof. Let $(G, \cdot)$ be a cyclic group of order n . Because G is cyclic, it is generated by some element $x \in G$, meaning $G = \langle x \rangle = \{e, x, x^2, \dots, x^{n-1}\}$.

We can map this group to the additive group of integers modulo n , denoted as $(\mathbb{Z}_n, +)$. Let $f : (\mathbb{Z}_n, +) \rightarrow (G, \cdot)$ be defined by $f(a) = x^a$. Because $f(a + b) = x^{a+b} = x^a \cdot x^b = f(a) \cdot f(b)$, the function f is a bijective homomorphism. Therefore, $G \cong \mathbb{Z}_n$.

Because an isomorphism perfectly preserves group structure, finding the number of generators in G is mathematically equivalent to finding the number of generators in $(\mathbb{Z}_n, +)$.

An element $m \in \mathbb{Z}_n$ generates the entire group $\mathbb{Z}_n$ if and only if repeated addition of m can produce the standard generator, 1. That is, there must exist some integer u such that:

$$m \cdot u \equiv 1 \pmod{n}$$

By the definition of modular congruence, this equation holds if and only if there exists another integer v such that:

$$m \cdot u + n \cdot v = 1$$

According to Bézout's Identity, this linear Diophantine equation has a solution for integers u and v if and only if the greatest common divisor of m and n is 1. Thus, m generates $\mathbb{Z}_n$ if and only if $\gcd(m, n) = 1$.

By definition, Euler's totient function, $\phi(n)$, counts the exact number of integers m in the range $1 \leq m \leq n$ that are coprime to n . Therefore, $(\mathbb{Z}_n, +)$ has exactly $\phi(n)$ generators. Due to the isomorphism $G \cong \mathbb{Z}_n$, the cyclic group G also possesses exactly $\phi(n)$ generators. $\square$

Part (b):

A Sophie-Germain prime P is such that $P - 1 = 2Q$, where Q is also prime. Q is typically called the Sophie-Germain prime while P is the "safe prime," but the structural relationship remains $P = 2Q + 1$.

Methodology : To properly generate a 512-bit prime P , the algorithm must carefully manage binary lengths. We begin by generating a random 511-bit odd integer Q . Specifically, we force the most significant bit to be 1, ensuring Q is strictly bounded by:

$$2^{510} \leq Q < 2^{511}$$

When we construct $P = 2Q + 1$, we can substitute the bounds of Q :

$$2(2^{510}) + 1 \leq 2Q + 1 < 2(2^{511}) + 1$$

$$2^{511} + 1 \leq P \leq 2^{512}$$

Because P must be an odd prime, it cannot equal exactly 2^{512} . Therefore, $2^{511} < P < 2^{512}$, mathematically guaranteeing that P is exactly 512 bits long.

Algorithm: Each candidate Q is subjected to our custom Miller-Rabin primality test (using 40 iterations for a negligible error margin). If Q passes, we compute P and subject it to the identical test. If P is also verified as prime, the algorithm halts.

(Refer to the output file for the explicit 512-bit value of P and 511-bit value of Q discovered during runtime.)

Part (c):

For a safe prime $P = 2Q + 1$, the multiplicative group of integers modulo P , denoted as $\mathbb{Z}_P^*$, has an order of:

$$|\mathbb{Z}_P^*| = \phi(P) = P - 1 = 2Q$$

According to Lagrange's Theorem, the order of any element $h \in \mathbb{Z}_P^*$ must perfectly divide the order of the group. Since Q is prime, the only possible divisors of $2Q$ are the set $\{1, 2, Q, 2Q\}$.

An element is a generator (primitive root) if and only if its order is exactly $2Q$. To computationally verify a candidate element h , we only need to ensure its order is not 1, 2, or Q . Because Q is extremely large, we can optimize the check by randomly selecting $h \in [2, P - 2]$ and ensuring:

1. $h^2 \not\equiv 1 \pmod{P}$ (Ensures the order is not 2)
2. $h^Q \not\equiv 1 \pmod{P}$ (Ensures the order is not Q)

Statistically, the number of generators in $\mathbb{Z}_P^*$ is $\phi(2Q) = \phi(2) \cdot \phi(Q) = Q - 1$. Since $Q \approx \frac{P}{2}$ (Euler's totient function), nearly 50% of all elements in the group are valid generators. Consequently, random sampling identifies a valid generator almost immediately.

Execution Verification: Our implemented program properly isolates a generator G using this probabilistic approach. It then verifies its correctness by computing $G^{P-1} \equiv 1 \pmod{P}$ while simultaneously ensuring $G^2 \not\equiv 1 \pmod{P}$ and $G^Q \not\equiv 1 \pmod{P}$.

(Pls Refer to the program output for the specific 512-bit generator value identified and its mathematical verification.)

Part (d):

Please check the code and output files for the main answer. Below is the theory behind the code:

The Diffie-Hellman protocol allows two entities to establish a shared cryptographic secret over an insecure channel using the parameters P (Safe Prime) and G (Generator of $\mathbb{Z}_P^*$).

Protocol :

1. **Entity A (Alice)** generates a secure random private key $a \in [2, P - 2]$. She computes the public broadcast $A \equiv G^a \pmod{P}$ and transmits A .
2. **Entity B (Bob)** generates a secure random private key $b \in [2, P - 2]$. He computes the public broadcast $B \equiv G^b \pmod{P}$ and transmits B .
3. **Derivation Phase:**
 - Alice receives B and computes the shared secret: $S_A \equiv B^a \pmod{P}$.
 - Bob receives A and computes the shared secret: $S_B \equiv A^b \pmod{P}$.

Mathematical Consistency & Security: The protocol guarantees $S_A = S_B$ because modular exponentiation is commutative:

$$S_A \equiv B^a \equiv (G^b)^a \equiv G^{ba} \equiv G^{ab} \equiv (G^a)^b \equiv A^b \equiv S_B \pmod{P}$$

The security of this exchange fundamentally relies on the Computational Diffie-Hellman (CDH) assumption. While an eavesdropper has full access to P, G, A , and B , deriving the value of G^{ab} is computationally intractable without first solving the Discrete Logarithm Problem (DLP) to extract a or b . This is the Diffie-Hellman Protocol as discussed in class.